

TARTU ÜLIKOOL
Arvutiteaduse instituut

**Relatsiooniliste ja dokumentorienteeritud
andmebaaside jõudluse võrdlus veebirakenduste
andmehalduses: PostgreSQL ja MongoDB analüüs**

Bakalaureusetöö (9 EAP)

Autor: Uku Renek

Juhendaja: Dr. Mati Kask, arvutiteaduse vanemteadur

Kaasjuhendaja: Tiina Tamm, MSc

Tartu 2026

Autorluse kinnitus

Kinnitan, et olen koostanud käesoleva töö iseseisvalt. Kõik töö koostamisel kasutatud teiste autorite tööd, põhimõttelised seisukohad, kirjandusallikatest ja mujalt pärinevad andmed on viidatud.

Autor: Uku Renek

Kuupäev: _____

Litsents

Annan Tartu Ülikoolile loa (lihtlitsents) enda loodud teose “Relatsiooniliste ja dokumentorienteeritud andmebaaside jõudluse võrdlus veebirakenduste andmehalduses: PostgreSQL ja MongoDB analüüs” reprodutseerimiseks eesmärgiga seda säilitada, sealhulgas lisada digitaalarhiivi DSpace kuni autoriõiguse kehtivuse tähtaja lõppemiseni.

Annan Tartu Ülikoolile loa teha punktis 1 nimetatud teos üldsusele kättesaadavaks Tartu Ülikooli veebikeskkonna, sealhulgas digitaalarhiivi DSpace kaudu Creative Commons litsentsiga CC BY NC ND 3.0, mis lubab autorile viidates teost reprodutseerida, levitada ja üldsusele suunata ning keelab luua tuletatud teost ja kasutada teost ärieesmärgil, kuni autoriõiguse kehtivuse tähtaja lõppemiseni.

Olen teadlik, et punktides 1 ja 2 nimetatud õigused jäävad alles ka autorile.

Autor: Uku Renek

Kuupäev: _____

Relatsiooniliste ja dokumentorienteeritud andmebaaside jõudluse võrdlus veebirakenduste andmehalduses: PostgreSQL ja MongoDB analüüs

Lühikokkuvõte

Käesolev bakalaureusetöö uurib relatsiooniline andmebaasihaldustarkvara PostgreSQL ja dokumentorienteeritud andmebaasihaldustarkvara MongoDB jõudlust ning sobivust erinevates veebirakenduste kontekstides. Töö eesmärk on anda terviklik ülevaade kahe erineva andmebaasiparadigma tugevatest ja nõrkadest külgedest ning pakkuda praktilisi juhiseid andmebaasi valikuks konkreetsete kasutusscenaariumide põhjal.

Uurimistöö raames viidi läbi süstemaatiline jõudlustestide seeria, mis hõlmas lugemis-, kirjutamis- ja keerukate päringute operatsioone erinevate andmemahutuste korral (1000 kuni 10 000 000 kirjet). Lisaks analüüsiti andmemudelite paindlikkust, skaleeritavust horisontaalselt ja vertikaalselt, tehingute tuge ning arendustöö keerukust.

Tulemused näitavad, et PostgreSQL on eelisseisus keerukate tehingute, andmete tervikluse tagamise ja mitme seotud tabeliga päringute korral. MongoDB osutub efektiivsemaks dokumentipõhiste andmete talletamisel, horisontaalsel skaleerumisel ja semistruktureeritud andmetega töötamisel. Hübriidsed arhitektuurilahendused, kus mõlemat süsteemi kasutatakse nende tugevuste järgi, annavad sageli parimaid tulemusi keerukates tootmiskeskkondades.

Töö pakub konkreetseid otsustuspuul põhinevad soovitusel andmebaasi valimiseks ning toob välja tulevikusuundumused polüglotse andmesäilituse valdkonnas.

Võtmesõnad: andmebaasid, PostgreSQL, MongoDB, jõudlus, relatsiooniline andmemudel, dokumentorienteeritud andmemudel, NoSQL, veebirakendused, skaleeritavus, andmehaldus

CERCS kood: P175 Informaatika, süsteemiteooria

Performance Comparison of Relational and Document-Oriented Databases in Web Application Data Management: Analysis of PostgreSQL and MongoDB

Abstract

This bachelor's thesis investigates the performance and suitability of the relational database management system PostgreSQL and the document-oriented database management system MongoDB in various web application contexts. The aim of this work is to provide a comprehensive overview of the strengths and weaknesses of two different database paradigms and to offer practical guidelines for database selection based on specific use case scenarios.

As part of the research, a systematic series of performance tests was conducted, covering read, write, and complex query operations with varying data volumes (1,000 to 10,000,000 records). Additionally, data model flexibility, horizontal and vertical scalability, transaction support, and development complexity were analysed.

The results show that PostgreSQL has an advantage in complex transactions, ensuring data integrity, and queries involving multiple related tables. MongoDB proves more efficient for storing document-based data, horizontal scaling, and working with semi-structured data. Hybrid architectural solutions, where both systems are used according to their strengths, often yield the best results in complex production environments.

The thesis provides concrete decision-tree-based recommendations for database selection and highlights future trends in the field of polyglot persistence.

Keywords: databases, PostgreSQL, MongoDB, performance, relational data model, document-oriented data model, NoSQL, web applications, scalability, data management

CERCS code: P175 Informatics, systems theory

Contents

Lühendite ja terminite loend	9
1 Sissejuhatus	1
1.1 Uurimistöö taust ja motivatsioon	1
1.2 Uurimisküsimused	2
1.3 Töö eesmärgid	2
1.4 Töö struktuur	2
2 Teoreetiline taust	4
2.1 Relatsiooniline andmemudel	4
2.1.1 Ajalugu ja alused	4
2.1.2 Normaliseerimise printsiibid	5
2.1.3 ACID-omadused	5
2.2 Dokumentorienteeritud andmemudel	6
2.2.1 NoSQL-liikumise tekkimine	6
2.2.2 MongoDB arhitektuur ja andmemudel	6
2.2.3 BASE-omadused ja CAP-teoreem	7
2.3 PostgreSQL ülevaade	8
2.3.1 Ajalugu ja areng	8
2.3.2 Sisemine arhitektuur	9
2.3.3 PostgreSQL vs. MySQL	9
2.4 MongoDB ülevaade	9
2.4.1 Ajalugu ja areng	9
2.4.2 Sisemine arhitektuur	10
3 Seotud tööd	11
3.1 Varasemad võrdlusuuringud	11
3.1.1 Jõudluse võrdlused	11
3.1.2 Skaleeritavuse uuringud	11
3.1.3 Hübriidarendused	12
3.2 Käesoleva töö panus	12
4 Metoodika	13

4.1	Uurimismetoodika ülevaade	13
4.2	Testimiskeskkond	13
4.2.1	Riistvara	13
4.2.2	Tarkvara versioonid	14
4.2.3	Andmebaaside konfigureerimine	14
4.3	Andmekogumid	15
4.3.1	Kasutajaprofilide andmekogum	15
4.3.2	Sündmuste logimise andmekogum	16
4.3.3	Geograafiliste andmete andmekogum	17
4.4	Testide kirjeldus	17
4.4.1	Test T1: Lihtne kirjutamine (INSERT)	17
4.4.2	Test T2: Masskijutamine (Batch INSERT)	17
4.4.3	Test T3: Primaarvõtme alusel lugemine	17
4.4.4	Test T4: Kompleksne lugemine koos filtrite ja sortimisega	17
4.4.5	Test T5: Uuendamine (UPDATE)	19
4.4.6	Test T6: Konkurentsed päringud	19
5	Tulemused	20
5.1	Test T1: Lihtne kirjutamine	20
5.2	Test T2: Masskirjutamine	21
5.3	Test T3: Primaarvõtme alusel lugemine	21
5.4	Test T4: Kompleksne lugemine koos liitmistega	22
5.5	Test T5: Uuendamise operatsioonid	23
5.6	Test T6: Konkurentsed päringud	24
5.7	Jõudluse kokkuvõte	24
6	Arutelu	26
6.1	Vastused uurimisküsimustele	26
6.1.1	RK1: Jõudluserinevused	26
6.1.2	RK2: Kasutajasobivuse analüüs	26
6.1.3	RK3: Andmemudeli mõju arendusele	27
6.1.4	RK4: Hübriidarhitektuurid	27
6.2	Otsustuspuu andmebaasi valikuks	28
6.3	Piirangud ja ohud	28
6.3.1	Uurimistöö piirangud	28
6.3.2	Levinud vead andmebaasi valikul	29
7	Kokkuvõte ja tulevikuperspektiivid	30
7.1	Peamised järeldused	30
7.2	Soovitused praktikutele	31

7.3	Tulevikusuundumused	31
7.3.1	Konvergennts	31
7.3.2	Pilvepõhised lahendused	32
7.3.3	Suurandmed ja tehisintellekt	32
7.3.4	Serverless ja edge computing	32
7.4	Uurimistöö edasiarendused	32
Kirjanduse loend		33
A Testimisskriptid		35
A.1	PostgreSQL jõudlustestide Python-skript	35
B Andmebaaside konfiguratsiooni võrdlustabel		40
C Kasutatud testimisandmekogumite statistika		41

List of Figures

6.1	Lihtsustatud otsustuspuidandmebaasi valikuks	28
-----	--	----

List of Tables

2.1	ACID-omaduste kirjeldus	5
4.1	Testimisserveri konfiguratsioon	14
4.2	Kasutatavate tarkvarakomponentide versioonid	14
5.1	Test T1 tulemused – üksikute kirjete lisamise aeg (ms) 10 000 kirje jaoks	20
5.2	Test T1 tulemused MongoDB erinevate write concern seadetega . . .	21
5.3	Test T2 tulemused – 1 000 000 kirje lisamine partiidena (1000 kirjet/- partii), koguaeg sekundites	21
5.4	Test T3 tulemused – üksiku kirje lugemine primaarvõtme alusel, me- diaanlatentsus (ms)	22
5.5	Test T4 tulemused – komplekspäring kahe tabeli liitmisega, medi- aanaeg (ms)	22
5.6	Test T4 tulemused – MongoDB denormaliseeritud vs. normaliseeritud andmemudel	23
5.7	Test T5 tulemused – üksiku kirje uuendamine primaarvõtme alusel (ms)	23
5.8	Test T6 tulemused – 100 paralleelset klienti, segatoimingute segu (70% lugemine, 30% kirjutamine)	24
5.9	Jõudlustestide koondtulemused	25
B.1	PostgreSQL ja MongoDB põhifunktsioonide võrdlus	40
C.1	Testimisandmekogumite kirjeldus	41

Lühendite ja terminite loend

Lühend/Termin	Selgitus
ACID	Atomaarsus, Järjepidevus, Isolatsioon, Vastupidavus (Atomicity, Consistency, Isolation, Durability)
API	Rakendusprogrammeerimisliides (Application Programming Interface)
BASE	Põhiliselt kättesaadav, Pehme olekutega, Lõpuks järjepidev (Basically Available, Soft state, Eventually consistent)
BSON	Binaar-JSON (Binary JSON) – MongoDB-s kasutatav andmeformaad
CAP	Järjepidevus, Kättesaadavus, Partitsioonitaluvus (Consistency, Availability, Partition tolerance)
CRUD	Loomine, Lugemine, Uuendamine, Kustutamine (Create, Read, Update, Delete)
DDL	Andmemääratluskeel (Data Definition Language)
DML	Andmemaniupuleerimiskeel (Data Manipulation Language)
DBMS	Andmebaasihaldussüsteem (Database Management System)
JSON	JavaScript objektide märkimisviis (JavaScript Object Notation)
MVCC	Mitmeversioonide konkurentsikontroll (Multi-Version Concurrency Control)
NoSQL	Mitte ainult SQL (Not Only SQL)
ORM	Objekt-relatsiooniline kujutamine (Object-Relational Mapping)
RAM	Muutmälu (Random Access Memory)

RDBMS	Relatsiooniline andmebaasihaldussüsteem (Relational Database Management System)
REST	Esitusseisundi ülekanne (Representational State Transfer)
SQL	Struktureeritud päringukeel (Structured Query Language)
SSD	Pooljuhtdraiv (Solid State Drive)
TPS	Tehinguid sekundis (Transactions Per Second)
WAL	Eelkirjutuslog (Write-Ahead Logging)

Chapter 1

Sissejuhatus

1.1 Uurimistöö taust ja motivatsioon

Andmebaaside valik on üks olulisemaid arhitektuurseid otsuseid iga tarkvarasüsteemi arendamisel. Väärvaliku tagajärjed võivad ilmnedas alles siis, kui süsteem on juba tootmiskeskkonnas ning andmemaht on kasvanud suurusjärke. Arendajad ja arhitektid seisavad regulaarselt valiku ees: kas kasutada aastakümnetepikkuse ajalooga relatsioonilist andmebaasihaldussüsteemi (RDBMS) või NoSQL-lahendust, mis lubab suuremat paindlikkust ja horisontaalset skaleeritavust?

Relatsiooniline andmebaasiparadigma, mille teoreetilised alused sõnastas Edgar F. Codd 1970. aastal [6], on tänaseni domineeriv lähenemine struktureeritud andmete talletamiseks. PostgreSQL, mis on üks võimsamaid avatud lähtekoodiga relatsioonilist andmebaasihaldussüsteeme, on laialdaselt tuntud oma ACID-vastavuse, laiendatavuse ning keerukate päringute toetuse poolest.

Samal ajal on NoSQL-andmebaaside populaarsus alates 2000. aastate lõpust märkimisväärselt kasvanud. Suured tehnoloogiaettevõtted nagu Google, Amazon ja Facebook seisid silmitsi andmemahtudega, mida traditsioonilised relatsioonilised süsteemid efektiivselt hallata ei suutnud. See sundis neid välja töötama alternatiivseid lahendusi, millest sai NoSQL-liikumise algus. MongoDB, mis ilmus turule 2009. aastal, on saanud üheks populaarseimaks dokumentorienteeritud andmebaasiks ning seda kasutatakse laialdaselt nii idufirmades kui suurtes ettevõtetes.

Vaatamata arvukatele turundusmaterjalidele ja osapoolt eelistavatele blogiartiklitele, puudub eestikeelne akadeemiline ülevaade, mis süstemaatiliselt võrdleks neid kahte süsteemi mitmesuguste reaalsete kasutusstsenaariumide korral. Käesolev töö täidab selle lünka.

1.2 Uurimisküsimused

Käesolev bakalaureusetöö otsib vastuseid järgmistele uurimisküsimustele:

1. **RK1:** Kuidas erinevad PostgreSQL ja MongoDB lugemis- ja kirjutamisjõudlus erinevate andmemahtude korral?
2. **RK2:** Millistes kasutusstsenaariumides on kumbki andmebaasiparadigma selgelt eelistatavam?
3. **RK3:** Kuidas mõjutavad andmemudeli erinevused arendustöö keerukust ja rakenduste hooldatavust?
4. **RK4:** Millised arhitektuursed mustrid võimaldavad mõlema paradigma tugevuste kombineerimist?

1.3 Töö eesmärgid

Bakalaureusetöö peamised eesmärgid on:

- Anda süstemaatiline teoreetiline ülevaade relatsioonilistest ja dokumentorienteeritud andmebaasiparadigmadest;
- Viia läbi kontrollitud jõudlustestid PostgreSQL ja MongoDB vahel standardsetes CRUD-operatsioonides;
- Analüüsida mõlema süsteemi käitumist erinevate andmemahtude ja päringumustrite korral;
- Koostada praktilised soovitused andmebaasi valikuks erinevate kasutusstsenaariumide põhjal;
- Tuua välja tulevikusuundumused andmebaaside valdkonnas.

1.4 Töö struktuur

Käesolev töö on üles ehitatud järgmiselt. Peatükis 2 antakse teoreetiline taust mõlemale andmebaasiparadigmale, selgitatakse ACID- ja BASE-omadusi ning kirjeldatakse CAP-teoreemi tähtsust hajutatud süsteemides. Peatükis 3 vaadeldakse varasemaid sarnaseid uurimusi ning paigutatakse käesolev töö laiemasse teaduslikku konteksti. Peatükis 4 kirjeldatakse uurimismetoodikat, testimiskeskonda ning kasutatavaid andmekogumeid. Peatükis 5 esitatakse jõudlustestide tulemused ning analüüsitakse neid. Peatükis 6 arutletakse tulemuste üle, vastatakse uurim-

isküsimustele ja pakutakse praktilisi soovitusi. Peatükk 7 koondab töö peamised järeldused ja tulevikuperspektiivid.

Chapter 2

Teoreetiline taust

2.1 Relatsiooniline andmemudel

2.1.1 Ajalugu ja alused

Relatsiooniline andmemudel sai alguse Edgar F. Codd 1970. aastal avaldatud revolutsioonilisest artiklist “A Relational Model of Data for Large Shared Data Banks” [6]. Codd pakkus välja matemaatiliselt formaalse aluse andmete organiseerimiseks, kasutades relatsiooniteooria mõisteid. Mudeli tuumaks on idee, et andmeid saab esitada kahemõõtmeliste tabelitena (relatsioonidena), kus iga rida esindab üht kirjet (tuplet) ja iga veerg ühte atribuuti.

1970.–1980. aastatel löid IBM Research ja Oracle esimesed kommertsiaalsed relatsioonilised andmebaasisüsteemid. SQL (Structured Query Language) standardiseeriti 1986. aastal (ANSI SQL) [7], mis tagas andmebaasisüsteemide vahelise teatud ühilduvuse ning soodustas relatsioonilise mudeli laialdast levikut.

Relatsioonilise mudeli põhikontseptid on:

- **Tabel (relatsioon):** Andmeid hoitakse tabelites, mis koosnevad ridadest ja veergudest. Iga tabel esindab üht olemitüüpi.
- **Primaarvõti:** Iga tabelis peab olema veerg või veergude kombinatsioon, mis identifitseerib iga rea üheselt.
- **Võõrvõti:** Mehhanism tabelitevaheli seoste loomiseks, mis tagab viitelise tervikluse.
- **Normaliseerimine:** Protsess andmete struktuurimiseks eesmärgiga vähendada andmete kordumist ja parandada terviklust.

2.1.2 Normaliseerimise printsiibid

Normaliseerimise teooria defineerib hulga normaalkujusid (NF – Normal Form), millest igaüks kõrvaldab teatud tüüpi anomaaliaid. Praktilises töös kasutatakse sageli kuni kolmandat normaalkuju (3NF) või Boyce-Coddi normaalkuju (BCNF) [7].

Esimene normaalkuju (1NF): Iga veerg sisaldab aatomilist (jagamatu) väärtust ning pole korduvaid veerugruppe.

Teine normaalkuju (2NF): Tabel on 1NF-is ning iga mitte-primaarvõtme atribuut sõltub täielikult primaarivõtimest (mitte ainult osast sellest).

Kolmas normaalkuju (3NF): Tabel on 2NF-is ning pole transitiivseid sõltuvusi primaarivõtmele mitte-primaarvõtme atribuutide kaudu.

Normaliseeritud skeem vähendab andmete kordumist, kuid suurendab päringute keerukust, kuna seotud andmeid tuleb sageli liita (JOIN) mitmest tabelist.

2.1.3 ACID-omadused

ACID on akronüüm, mis kirjeldab relatsioonilist andmebaaside tehingute nelja põhiomadust [12]:

Table 2.1: ACID-omaduste kirjeldus

Omadus	Kirjeldus
Atomaarsus	Tehing täidetakse täielikult või ei täideta üldse. Kui ühegi sammu täitmine ebaõnnestub, tühistatakse kõik muutused.
Järjepidevus	Tehing viib andmebaasi ühest kehtivast olekust teise. Kõik tervikluspiirangud peavad olema täidetud nii enne kui pärast tehingut.
Isolatsioon	Paralleelsed tehingud ei mõjuta üksteist. Iga tehing käitub nii, nagu ta oleks süsteemis ainus.
Vastupidavus	Kui tehing on kinnitatud (committed), jäävad muutused püsima ka süsteemitõrgete korral.

ACID-omadused on eriti kriitilised rakendustes, kus andmete terviklus on esmatähtis – näiteks pangandustarkvara, varude haldus ja tervishoiusüsteemid.

2.2 Dokumentorienteeritud andmemudel

2.2.1 NoSQL-liikumise tekkimine

NoSQL-liikumine sai hoo sisse 2000. aastate teisel poolel, kui suured tehnoloogiaettevõtted nagu Google ja Amazon seisis silmitsi probleemidega, mida traditsioonilised relatsioonilised andmebaasid lahendada ei suutnud [15]. Google avaldas 2006. aastal Bigtable'i kirjelduse [4] ning Amazon DynamoDB kontseptsioon avaldati 2007. aastal [8]. Need süsteemid pidid hakkama toime väga suure andmemahuga, kõrge samaaegsusega ning nõudsid kõrget kättesaadavust.

Termin “NoSQL” ei tähenda “pole SQL-i”, vaid pigem “mitte ainult SQL” – see viitab andmebaasisüsteemide perekonnale, mis ei järgi rangelt relatsioonilist mudelit [9]. NoSQL-andmebaasid jagunevad peamiselt nelja kategooriasse:

1. **Dokumentandmebaasid:** MongoDB, CouchDB, Firestore
2. **Võti-väärtus poed:** Redis, DynamoDB, Riak
3. **Veeruandmebaasid:** Cassandra, HBase, Scylla
4. **Graafandmebaasid:** Neo4j, Amazon Neptune, ArangoDB

2.2.2 MongoDB arhitektuur ja andmemudel

MongoDB on 2009. aastal asutatud ettevõtte MongoDB Inc. poolt välja töötatud dokumentorienteeritud andmebaas. Andmeid talletatakse JSON-sarnases BSON-formaadis (Binary JSON), mis võimaldab keerukaid pesastatud struktuure [5].

MongoDB põhikontseptid on:

- **Dokument:** JSON-objektile vastav andmeühik, mis võib sisaldada väärtusi, massiive ja pesastatud dokumente. Vastab ligikaudu relatsioonilise mudeli reale.
- **Kogu (Collection):** Dokumentide konteiner. Vastab ligikaudu relatsioonilise mudeli tabelile, kuid ei nõua fikseeritud skeemi.
- **Andmebaas:** Kogude konteiner. Üks MongoDB eksemplar võib sisaldada mitmeid andmebaase.
- **Indeks:** Andmestruktuur, mis kiirendab päringuid valitud välja(de) alusel.

MongoDB dokumendi näide:

```
1 {  
2   "_id": ObjectId("64a7f3e2b1c2d3e4f5a6b7c8"),
```

```
3  "kasutajanimi": "jaan.tamm",
4  "email": "jaan.tamm@example.com",
5  "nimi": {
6    "eesnimi": "Jaan",
7    "perenimi": "Tamm"
8  },
9  "vanus": 28,
10 "registreerimisKuup ev": ISODate("2024-01-15T10:30:00Z"),
11 "rollid": ["kasutaja", "toimetaja"],
12 "eelistused": {
13   "keel": "et",
14   "teema": "tume",
15   "teavitused": true
16 },
17 "ostud": [
18   {
19     "tooteId": "prod_001",
20     "nimetus": "Klaviatuur",
21     "hind": 89.99,
22     "kuup ev": ISODate("2024-03-20T14:22:00Z")
23   },
24   {
25     "tooteId": "prod_002",
26     "nimetus": "Hiir",
27     "hind": 45.50,
28     "kuup ev": ISODate("2024-05-01T09:15:00Z")
29   }
30 ]
31 }
```

Listing 2.1: MongoDB dokumendi näide (kasutajaprofiil)

2.2.3 BASE-omadused ja CAP-teoreem

Erinevalt ACID-ist kasutavad paljud NoSQL-süsteemid BASE-mudelit [2]:

- **Põhiliselt kättesaadav (Basically Available):** Süsteem garanteerib kättesaadavuse CAP-teoreemi tähenduses.
- **Pehme olekutega (Soft state):** Süsteemi olek võib ajas muutuda isegi ilma sisendita, kuna lõpuks-järjepideva mudeli tõttu.
- **Lõpuks järjepidev (Eventually consistent):** Süsteem saavutab järjepide-

vuse lõpuks, kui sellele anda piisavalt aega.

CAP-teoreem, mille sõnastas Eric Brewer 2000. aastal [2] ja mida tõestasid Gilbert ja Lynch 2002. aastal [10], väidab, et hajutatud süsteem saab korraga garanteerida ainult kaks järgmisest kolmest omadusest:

- **Järjepidevus (Consistency):** Kõik sõlmed näevad üheaegselt samu andmeid.
- **Kättesaadavus (Availability):** Iga päring saab vastuse, isegi kui mõned sõlmed on mahavõetud.
- **Partitsioonitaluvus (Partition tolerance):** Süsteem toimib edasi isegi siis, kui võrguühendus sõlmede vahel katkeb.

Kuna reaalsetes hajutatud süsteemides on võrgupartitsioonid vältimatud, peavad süsteemid valima järjepidevuse ja kättesaadavuse vahel. PostgreSQL eelistab järjepidevust (CP-süsteem), MongoDB aga pakub paindlikku valikut, vaikumisi kaldudes kättesaadavuse poole (AP-süsteem).

2.3 PostgreSQL ülevaade

2.3.1 Ajalugu ja areng

PostgreSQL on avatud lähtekoodiga relatsiooniline andmebaasihaldussüsteem, mille juured ulatuvad Berkeley ülikoolis 1986. aastal alustatud POSTGRES-projekti [16]. PostgreSQL, nagu seda täna tuntakse, sai alguse 1994. aastal, kui Andrew Yu ja Jolly Chen lisasid SQL-i tugi. Alates 1996. aastast areneb see globaalse vabatahtlike kogukonna poolt.

PostgreSQL eristab end teistest RDBMS-idest mitme olulise omaduse poolest:

- Täielik ACID-vastavus
- Laiendatav tüübisüsteem (kasutajaloodud tüübid, operaatorid, funktsioonid)
- Mitmeversioonide konkurentsikontroll (MVCC)
- Täielik SQL-standardi tugi, sealhulgas aknaoperatsioonid (window functions)
- JSON/JSONB andmetüüpide tugi (alates versioonist 9.2/9.4)
- Täistekstiotsing, geograafilised andmed (PostGIS)
- Tabelipärandus ja osatabelid

2.3.2 Sisemine arhitektuur

PostgreSQL kasutab protsessipõhist arhitektuuri: iga kliendiühendus loob eraldi serverprotsessi. See on erinevalt mõnest muust süsteemist (nt MySQL), mis kasutab lõimepõhist mudelit.

Olulised sisemised komponendid:

- **Shared Buffer Pool:** Mälus hoitav puhver andmelehe vahemälu jaoks. Jõudluse jaoks on kriitiline, et see oleks piisavalt suur.
- **WAL (Write-Ahead Log):** Kõik andmemuutused logitakse enne kettale kirjutamist, tagades töökindluse.
- **MVCC:** Iga rida võib eksisteerida mitmes versioonis, võimaldades lugejatel töötada ilma kirjutajaid blokeerimata.
- **Päringute optimeerija:** Statistikapõhine kuluoptimeerija valib välja efektiivseima täitmisplaani.
- **Indeksitüübid:** B-puu, hash, GiST, SP-GiST, GIN, BRIN.

2.3.3 PostgreSQL vs. MySQL

Kuigi käesolev töö keskendub PostgreSQL ja MongoDB võrdlusele, on oluline mainida, et PostgreSQL valikul relatsioonilise kandidaadina võrreldi seda MySQL/MariaDB-ga. PostgreSQL valiti järgmistel põhjustel:

- Rangem ACID-vastavus (MySQL InnoDB on samuti ACID-ühilduv, kuid PostgreSQL ajalooliselt rangem)
- Parem JSON-tugi (JSONB tüüp koos täielike indekseerimise võimalustega)
- Keerukate päringute parem jõudlus (common table expressions, window functions)
- Laialdane kasutus teaduslikes võrdlusuuringutes

2.4 MongoDB ülevaade

2.4.1 Ajalugu ja areng

MongoDB arendas algselt ettevõtte 10gen (hiljem ümber nimetatud MongoDB Inc.), kes avaldas esimese versiooni 2009. aastal [5]. Nimi “MongoDB” tuleneb sõnast “humongous” (tohtu), mis vihjab süsteemi võimekusele suurt andmemahtu hallata.

MongoDB arengus on mitu olulist verstapostverstapost:

- **2009:** Esimene avalik väljalase
- **2012 (v2.4):** Tekst-indeksite ja agregatsioonikonveieri lisamine
- **2015 (v3.0):** WiredTiger salvestusmootori lisamine, mis parandas märkimisväärselt kirjutusjõudlust ja kokkusurumist
- **2018 (v4.0):** Mitmesuse tehingute (multi-document ACID transactions) tugi
- **2019 (v4.2):** Hajutatud tehingud
- **2021 (v5.0):** Ajaseriad, versiooniuuendused
- **2023 (v7.0):** Täiendavad ACID-täiustused, jõudluse parandused

2.4.2 Sisemine arhitektuur

MongoDB kasutab alates versioonist 3.2 vaikimisi WiredTiger salvestusmootorit, mis pakub:

- Dokumenditaseme lukustamist (varasemates versioonides oli kogutaseme lukustamine)
- Andmete pakkimist (Snappy, zlib, zstd)
- Kontrollpunkte iga 60 sekundi järel
- Ajakirja (journal) kirjutamist iga 50 millisekundi järel

MongoDB horisontaalne skaleerumine toimub jagamise (sharding) kaudu:

- **Shard:** Üks osa andmete kogumist, mis võib olla replika hulk.
- **Config server:** Hoiab metaandmeid jagamise kohta.
- **mongos:** Päringute marsruuter, mis suunab päringud õigesse shardisse.

Replika hulk (replica set) koosneb ühest primaarsest sõlmest ja mitmest sekundaarsest sõlmest. Primaarset sõlm vastuvõttab kõik kirjutamisoperatsioonid, sekundaarsed sõlmed replitseerivad andmeid ning saavad teenindada lugemisoperatsioone.

Chapter 3

Seotud tööd

3.1 Varasemad võrdlusuuringud

Andmebaaside jõudluse võrdlus on olnud akadeemilise uurimistöö teema juba mõne aasta. Siinses peatükis tuuakse välja olulisemad seotud uurimused ning nende peamised leiud.

3.1.1 Jõudluse võrdlused

Abramova ja Bernardino [1] viisid 2013. aastal läbi MongoDB, Cassandra ja MySQL võrdluse, kasutades YCSB (Yahoo! Cloud Serving Benchmark) testimisraamistikku. Nende tulemused näitasid, et MongoDB ületas MySQL-i lugemisoperatsioonides, kuid MySQL oli üldam mõningate kirjutamisstsenaariumide korral parema järjepidevuse tõttu.

Györödi jt [11] võrdlesid 2015. aastal MongoDB ja MySQL jõudlust e-kaubanduse stsenaariumi kontekstis. Uurijad leidsid, et MongoDB ületas MySQL-i suurte andme-
hulkade korral kirjutamisoperatsioonides, samas kui MySQL oli täpsema tulemuse andmisel parem keerukate JOINide kasutamisel.

Kaur ja Rani [13] andsid ülevaate SQL ja NoSQL andmebaaside erinevustest ning jõudsid järeldusele, et kummagi süsteemi valik sõltub suuresti rakenduse nõuetest: struktureeritud andmete jaoks sobib SQL paremini, semistruktureeritud suure mahuga andmete korral NoSQL.

3.1.2 Skaleeritavuse uuringud

Cattell [3] uuris erinevate NoSQL-andmebaaside skaleeritavust ja jõudis järeldusele, et NoSQL-süsteemid pakuvad paremaid võimalusi horisontaalseks skaleerimiseks.

Siiski rõhutas autor, et paljud NoSQL-süsteemid saavutasid paremad tulemused nõrgemate konsistentsustagatiste tõttu.

Leavitt [14] analüüsis NoSQL-andmebaaside kasutamise trende ning järeldas, et NoSQL-süsteemid sobivad hästi pilvekeskkondadesse ja suurandmetega töötamiseks, kuid traditsioonilised ärirakendused jäävad suures osas sõltuma relatsioonilistest andmebaasidest.

3.1.3 Hübriidarendused

Fowler ja Sadalage [9] tutvustasid kontseptsiooni “polyglot persistence” – mõte, et üks rakendus võib kasutada erinevaid andmebaasilahendusi erinevate andmetüüpide ja kasutusmustrite jaoks. See lähenemine on saanud oluliseks praktikaks mikroteenuste arhitektuurides.

3.2 Käesoleva töö panus

Võrreldes varasemate uurimustega erineb käesolev töö mitmel viisil:

1. **Keeleline panus:** Eestikeelne akadeemiline ülevaade on selles valdkonnas puudulik, mis muudab selle töö kohalikuks referentsiks.
2. **Uuemad versioonid:** Varasemates uuringutes kasutati sageli MongoDB versioone enne mitmesuse tehingute lisamist (v4.0). Käesolevas töös kasutatakse MongoDB 7.x ja PostgreSQL 16.x.
3. **Praktilised stsenaariumid:** Töö keskendub konkreetsetele veebirakendustele tüüpilistele muster, mitte ainult sünteetilistele testidele.
4. **Otsustusraamistik:** Pakutakse struktureeritud otsustuspuu, mis aitab arendajatel andmebaasi valida konkreetsete nõuete põhjal.

Chapter 4

Metoodika

4.1 Uurimismetoodika ülevaade

Käesolev uurimistöö kasutab eksperimentaalset lähenemist. Jõudluse võrdlemiseks viidi läbi kontrollitud testid standardsetes tingimustes, kasutades nii sünteetilisi testide andmeid kui ka reaalse rakenduse mustreid imiteerivaid andmekogumeid.

Uurimismetoodika koosneb kolmest etapist:

1. **Etapp 1 – Testimiskeskonna seadistamine:** Mõlemad andmebaasisüsteemid paigaldati identsetele serveritele identse konfiguratsiooni ja riistvaraga.
2. **Etapp 2 – Andmekogumite loomine:** Koostati viis erinevat andmekogumit suurusega 1000 kuni 10 000 000 kirjet.
3. **Etapp 3 – Jõudlustestid:** Iga test käivitati 10 korda, et vähendada mõõtemüra mõju. Tulemuste esitamisel kasutatakse mediaani ja standardhälvet.

4.2 Testimiskeskond

4.2.1 Riistvara

Kõik testid viidi läbi kahes identses virtuaalmasinas järgmise konfiguratsiooniga:

Table 4.1: Testimisserveri konfiguratsioon

Komponent	Spetsifikatsioon
Protsessor	Intel Core i7-12700K, 12 tuuma, 3.6 GHz
RAM	32 GB DDR4-3200
Salvestusruum	1 TB NVMe SSD (Samsung 980 Pro)
Võrk	10 Gbps sisevõrk
Operatsioonisüsteem	Ubuntu 22.04 LTS

4.2.2 Tarkvara versioonid

Table 4.2: Kasutatavate tarkvarakomponentide versioonid

Tarkvara	Versioon
PostgreSQL	16.2
MongoDB	7.0.5
Python	3.11.4
psycopg2	2.9.9
pymongo	4.6.1
pgbench	16.2

4.2.3 Andmebaaside konfigureerimine

Andmebaaside seadistamisel lähtuti igale süsteemile sobivatest tootmiskeskonna soovitustest.

PostgreSQL konfiguratsiooni olulisemad parameetrid:

```

1 # M lu seaded
2 shared_buffers = 8GB           # 25% serveri RAMist
3 effective_cache_size = 24GB    # 75% serveri RAMist
4 work_mem = 256MB              # Sorteerimise ja hashimise
    m lu
5 maintenance_work_mem = 2GB    # VACUUM ja indekse
    ehitamise m lu
6
7 # Kontrollpunkti seaded
8 checkpoint_completion_target = 0.9
9 wal_buffers = 64MB
10
```

```

11 # Parallelism
12 max_parallel_workers_per_gather = 4
13 max_parallel_workers = 8
14
15 # I/O seaded
16 random_page_cost = 1.1           # NVMe SSD jaoks
    optimeeritud
17 effective_io_concurrency = 200    # NVMe SSD jaoks

```

Listing 4.1: PostgreSQL konfiguratsioonifail (postgresql.conf olulisemad seaded)

MongoDB konfiguratsiooni olulisemad parameetrid:

```

1 storage:
2   dbPath: /var/lib/mongodb
3   wiredTiger:
4     engineConfig:
5       cacheSizeGB: 12
6     collectionConfig:
7       blockCompressor: snappy
8     indexConfig:
9       prefixCompression: true
10
11 operationProfiling:
12   slowOpThresholdMs: 100
13
14 net:
15   port: 27017
16   bindIp: 127.0.0.1

```

Listing 4.2: MongoDB konfiguratsioonifail (mongod.conf)

4.3 Andmekogumid

4.3.1 Kasutajaprofilide andmekogum

Esimene andmekogum simuleerib veebipoes kasutatavaid kasutajaprofiile. See andmekogum sisaldab erinevaid andmetüüpe, sealhulgas stringid, arvud, kuupäevad ja massiivid.

PostgreSQL skeem:

```

1 CREATE TABLE kasutajad (

```

```

2      id          SERIAL PRIMARY KEY,
3      kasutajanimi VARCHAR(50) UNIQUE NOT NULL,
4      email       VARCHAR(255) UNIQUE NOT NULL,
5      eesnimi     VARCHAR(100) NOT NULL,
6      perenimi    VARCHAR(100) NOT NULL,
7      vanus       SMALLINT CHECK (vanus >= 0 AND vanus <= 150),
8      reg_kuup ev  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
9      aktive      BOOLEAN NOT NULL DEFAULT TRUE,
10     meta        JSONB
11 );
12
13 CREATE TABLE kasutaja_rollid (
14     kasutaja_id INTEGER REFERENCES kasutajad(id) ON DELETE
15     CASCADE,
16     roll         VARCHAR(50) NOT NULL,
17     PRIMARY KEY (kasutaja_id, roll)
18 );
19
20 CREATE TABLE ostud (
21     id          SERIAL PRIMARY KEY,
22     kasutaja_id INTEGER REFERENCES kasutajad(id) ON DELETE
23     CASCADE,
24     toode_id    VARCHAR(50) NOT NULL,
25     nimetus     VARCHAR(255) NOT NULL,
26     hind        NUMERIC(10, 2) NOT NULL CHECK (hind >= 0),
27     ostu_aeg    TIMESTAMPTZ NOT NULL DEFAULT NOW()
28 );
29
30 -- Indeksid
31 CREATE INDEX idx_kasutajad_email ON kasutajad(email);
32 CREATE INDEX idx_kasutajad_reg ON kasutajad(reg_kuup ev DESC
33 );
34 CREATE INDEX idx_ostud_kasutaja ON ostud(kasutaja_id);
35 CREATE INDEX idx_ostud_aeg ON ostud(ostu_aeg DESC);

```

Listing 4.3: PostgreSQL kasutajate tabelite skeem

4.3.2 Sündmuste logimise andmekogum

Teine andmekogum simuleerib sündmuste logisid, kus on iseloomulik suur kirjutamis-
maht ning varieeruv skeem sõltuvalt sündmuse tüübist.

4.3.3 Geograafiliste andmete andmekogum

Kolmas andmekogum sisaldab geograafilisi punkte (asukohad, koordinaadid), mis võimaldavad testida geograafiliste päringute jõudlust.

4.4 Testide kirjeldus

4.4.1 Test T1: Lihtne kirjutamine (INSERT)

Testiti üksikute kirjade lisamist ilma tehinguta. Mõõdeti aega, mis kulus 10 000 kirje lisamiseks.

4.4.2 Test T2: Masskiijutamine (Batch INSERT)

Testiti 1000-kirjeliste partiide lisamist. See simuleerib andmete importimist või ETL-protsesse.

4.4.3 Test T3: Primaarvõtme alusel lugemine

Testiti üksiku kirje toomist teadaoleva ID alusel. See on kõige levinuim päringumuster veebirakendustes.

4.4.4 Test T4: Kompleksne lugemine koos filtrite ja sortimisega

Testiti päringuid, mis hõlmavad WHERE-tingimusi, sortimist ja piiranguid. PostgreSQLis kasutati SQL-päringuid, MongoDBs aggregate pipeline'i.

```
1 SELECT
2     k.id,
3     k.kasutajanimi,
4     k.email,
5     k.eesnimi || ' ' || k.perenimi AS taisnimi,
6     COUNT(o.id) AS ostude_arv,
7     SUM(o.hind) AS kulutatud_kokku,
8     MAX(o.ostu_aeg) AS viimane_ost
9 FROM kasutajad k
10 LEFT JOIN ostud o ON o.kasutaja_id = k.id
11 WHERE
12     k.reg_kuup ev >= '2023-01-01'
13     AND k.aktive = TRUE
14     AND k.vanus BETWEEN 18 AND 35
```

```
15 GROUP BY k.id, k.kasutajanimi, k.email, k.eesnimi, k.perenimi
16 HAVING COUNT(o.id) > 0
17 ORDER BY kulutatud_kokku DESC
18 LIMIT 100;
```

Listing 4.4: PostgreSQL komplekspäring (Test T4)

```
1 db.kasutajad.aggregate([
2   {
3     $match: {
4       reg_kuup ev: { $gte: ISODate("2023-01-01") },
5       aktive: true,
6       vanus: { $gte: 18, $lte: 35 }
7     }
8   },
9   {
10    $lookup: {
11      from: "ostud",
12      localField: "_id",
13      foreignField: "kasutaja_id",
14      as: "ostud"
15    }
16  },
17  {
18    $addFields: {
19      ostude_arv: { $size: "$ostud" },
20      kulutatud_kokku: { $sum: "$ostud.hind" },
21      viimane_ost: { $max: "$ostud.ostu_aeg" },
22      taisnimi: { $concat: ["$eesnimi", " ", "$perenimi"] }
23    }
24  },
25  {
26    $match: { ostude_arv: { $gt: 0 } }
27  },
28  {
29    $sort: { kulutatud_kokku: -1 }
30  },
31  {
32    $limit: 100
33  },
34  {
35    $project: {
```

```
36     kasutajanimi: 1,  
37     email: 1,  
38     taisnimi: 1,  
39     ostude_arv: 1,  
40     kulutatud_kokku: 1,  
41     viimane_ost: 1  
42 }  
43 }  
44 ])
```

Listing 4.5: MongoDB ekvivalentne aggregatsioonipäring (Test T4)

4.4.5 Test T5: Uuendamine (UPDATE)

Testiti üksiku kirje uuendamist teadaoleva primaarvõtme alusel ning mitme kirje korraga uuendamist.

4.4.6 Test T6: Konkurentsed päringud

Simuleeriti 100 paralleelset kliendiühendust, millest igaüks täitis vaheldumisi loe- ja kirjutamisoperatsioone. See test mõõdab läbilaskevõimet (TPS) ning keskmist latentsust.

Chapter 5

Tulemused

5.1 Test T1: Lihtne kirjutamine

Üksikute kirjete lisamise testis ilmnas MongoDB eelis väiksemate andmemahtude korral. Tabel 5.1 esitab mediaanajad 10 000 kirje lisamiseks erinevate andmemahtude (olemasolevate andmete arv) korral.

Table 5.1: Test T1 tulemused – üksikute kirjete lisamise aeg (ms) 10 000 kirje jaoks

Olemaolev andmemaht	PostgreSQL (ms)	MongoDB (ms)	Erinevus (%)
1 000 kirjet	1 243	892	−28,2%
10 000 kirjet	1 389	967	−30,4%
100 000 kirjet	1 521	1 034	−32,0%
1 000 000 kirjet	1 687	1 123	−33,4%
10 000 000 kirjet	2 134	1 356	−36,5%

MongoDB kiirus kirjutamisoperatsioonides tuleneb osaliselt sellest, et vaikimisi ei oota MongoDB kirjutamisoperatsioon kettal salvestamist ära (write concern “w:1” tagab ainult mälu olevate andmete kinnituse). PostgreSQL, olles rangemalt ACID-vastavuses, tagab iga tehinguga WAL-logisse kirjutamise, mis lisab latentsust.

Kui MongoDB konfigureeritakse kõrgeima write concern’iga (“j: true”, mis nõuab ajakirja sünkroonimist), kaovad jõudluserinevused suures osas:

Table 5.2: Test T1 tulemused MongoDB erinevate write concern seadetega

Write concern	Aeg (ms)	Andmekindluse tase
w:0 (fire and forget)	234	Minimaalne
w:1 (vaikimisi)	892	Mõõdukas
w:majority	1 089	Kõrge
w:1, j:true	1 678	Maksimaalne
PostgreSQL (synchronous_commit=on)	1 243	Maksimaalne

Järeldus T1: Sarnaste andmekindluse garantiide korral on PostgreSQL ja MongoDB kirjutusjõudlus sarnane. MongoDB eelis varasemates uuringutes tulenes sageli madalamast vaikimisi write concern seadest.

5.2 Test T2: Masskirjutamine

Partiikirjutamise testis (1000-kirjelised partiid) näitas PostgreSQL selgemat eelist tänu tehingute efektiivsemale käsitlemisele:

Table 5.3: Test T2 tulemused – 1 000 000 kirje lisamine partiidena (1000 kirjet/partii), koguaeg sekundites

Andmebaasimotor	Koguaeg (s)	Kiirus (kirjet/s)
PostgreSQL (COPY)	8.3	120 482
PostgreSQL (INSERT partiidega)	24.1	41 494
MongoDB (insertMany)	31.7	31 546
MongoDB (ordered: false)	19.2	52 083

PostgreSQL COPY käsk on eriti efektiivne suurte andmemahutude importimiseks, kuna see möödub päringtöötlejast ja kirjutab andmeid otse andmebaasi mootorisse.

5.3 Test T3: Primaarvõtme alusel lugemine

Üksiku kirje toomine primaarvõtme alusel on mõlema andmebaasi jaoks triviaalne operatsioon, millel peaks olema $O(1)$ keerukus, kui andmebaasil on vastav indeks.

Table 5.4: Test T3 tulemused – üksiku kirje lugemine primaarvõtme alusel, mediaan-latentsus (ms)

Andmemahu	PostgreSQL (ms)	MongoDB (ms)
1 000 kirjet	0.21	0.19
10 000 kirjet	0.22	0.20
100 000 kirjet	0.24	0.21
1 000 000 kirjet	0.31	0.25
10 000 000 kirjet	0.48	0.38

Mõlemad süsteemid näitavad kiiremat lugemist väiksemate andmemahtude korral, mil kõik indeksid mahuvad puhvrisesse. Erinevused on minimaalsed ja praktilises mõttes ebaolulised.

5.4 Test T4: Kompleksne lugemine koos liitmis- tega

See test näitas kõige olulisemaid erinevusi kahe süsteemi vahel. PostgreSQL on selgelt eelistatavam olukordades, kus andmed on normaliseeritud mitmesse tabelisse:

Table 5.5: Test T4 tulemused – komplekspäring kahe tabeli liitmisega, mediaanaeg (ms)

Andmemahu	PostgreSQL (ms)	MongoDB (ms)	Suhe PG/Mongo
10 000 kasutajat	12.4	87.3	7.04×
100 000 kasutajat	98.7	734.2	7.44×
1 000 000 kasutajat	1 023	8 921	8.72×

MongoDB mahajäämus on seotud sellega, et `$lookup` (liitmine) on MongoDB-s hiljem lisatud funktsioon ning ei ole nii optimeeritud kui PostgreSQL-i JOIN-operaator. Lisaks on MongoDB-s andmete denormaliseeritud talletamine sageli soovitatud just selle probleemi vältimiseks.

Kui andmeid talletada MongoDB-s denormaliseeritud kujul (ostud pesastatud kasutajas), on tulemus oluliselt parem:

Table 5.6: Test T4 tulemused – MongoDB denormaliseeritud vs. normaliseeritud andmemudel

MongoDB andmemudel	1M kasutajat (ms)	Suhe vs. PostgreSQL
Normaliseeritud (\$lookup)	8 921	8.72× aeglasem
Denormaliseeritud (pesastatud ostud)	1 247	1.22× aeglasem

5.5 Test T5: Uuendamise operatsioonid

Uuendamise testis olid tulemused sarnased. Üksiku kirje uuendamine on mõlemas süsteemis sarnase jõudlusega:

Table 5.7: Test T5 tulemused – üksiku kirje uuendamine primaarvõtme alusel (ms)

Andmemahu	PostgreSQL (ms)	MongoDB (ms)
1 000 kirjet	0.31	0.28
1 000 000 kirjet	0.42	0.35
10 000 000 kirjet	0.67	0.51

Mitme kirje korraga uuendamine oli samuti sarnane, kuid PostgreSQL-i tehingute tugi võimaldab keerukamat loogikat sama tehingu raames:

```

1 BEGIN;
2
3 -- Uuenda kasutaja andmeid
4 UPDATE kasutajad
5 SET aktive = FALSE,
6     meta = meta || '{"deaktiveerimise_aeg": "2026-01-15"}'
7 WHERE email = 'jaan.tamm@example.com';
8
9 -- Logi muutus
10 INSERT INTO audit_log (
11     tabel, operatsioon, kasutaja_id, muutused, aeg
12 )
13 SELECT
14     'kasutajad',
15     'DEAKTIVEERIMINE',
16     id,
17     jsonb_build_object('aktive', FALSE),
18     NOW()

```

```
19 FROM kasutajad
20 WHERE email = 'jaan.tamm@example.com';
21
22 COMMIT;
```

Listing 5.1: PostgreSQL tehingupõhine uuendamine

5.6 Test T6: Konkurentsed päringud

100 paralleelse kliendiga testi tulemused näitasid mõlema süsteemi käitumist suure koormuse all:

Table 5.8: Test T6 tulemused – 100 paralleelset klienti, segatoimingute segu (70% lugemine, 30% kirjutamine)

Andmebaasimotor	TPS	Mediaanlatentsus (ms)	p95 latentsus (ms)	p99 latent
PostgreSQL	18 742	4.2	11.3	
MongoDB	21 391	3.8	9.7	

MongoDB näitas mõõdukat eelist suure konkurentsuse korral lihtsate operatsioonide puhul, mis on seletatav selle dokumenditaseme lukustamisskeemiga ja WiredTiger-mootori optimeerimistega.

5.7 Jõudluse kokkuvõte

Allpool on esitatud koondülevaade kõigi testide tulemustest:

Table 5.9: Jõudlustestide koondtulemused

Test	PostgreSQL	MongoDB	Märkused
T1: Üksik INSERT	≈	+	MongoDB kiirem, kuid sarnane ACID-seadetega
T2: Partii INSERT	++	≈	PostgreSQL COPY eriti efektiivne
T3: Primaarvõtme lugemine	≈	≈	Mõlemad sarnased
T4: Kompleks JOIN	+++	–	PostgreSQL selgelt eelistatavam
T5: Üksik UPDATE	≈	≈	Sarnased tulemused
T6: Konkurentsed päringud	+	++	MongoDB parem läbi-laskevõime
+++ = tugevalt parem, ++ = märgatavalt parem, + = mõõdukalt parem, ≈ = sarnane, – =			

Chapter 6

Arutelu

6.1 Vastused uurimisküsimustele

6.1.1 RK1: Jõudluserinevused

Jõudlustestide tulemused näitasid, et kummagi süsteemi selge eelis sõltub oluliselt kasutusstsenaariumist. PostgreSQL on eelisseisus keerukates päringuoperatsioonides, mis hõlmavad mitme tabeli liitmisi. MongoDB näitas eeliseid lihtsate kirjutamisoperatsioonide ja konkurentsete lihtsate päringute korral.

Oluline leid on, et paljud varasemad MongoDB jõudluseelised kaduvad, kui mõlemad süsteemid konfigureeritakse sarnaste andmekindluse garantiidega. See on kooskõlas Stonebraker jt [17] kriitikaga NoSQL-süsteemide jõudlusväidete kohta, mis ignoreerivad konsistentsusnõudeid.

6.1.2 RK2: Kasutajasobivuse analüüs

Uurimistulemuste põhjal saab eristada järgmisi kasutusstsenaariumide kategooriaid:

PostgreSQL on selgelt eelistatav järgmistel juhtudel:

- Rahalised tehingud, kus ACID-garantiid on kriitilised
- Keerukad aruanded, mis hõlmavad mitme tabeli liitmisi
- Tugevalt struktureeritud andmed kindla skeemiga
- Rakendused, kus andmete terviklus on esmatähtis (tervishoid, pangandus)
- Olemasolevate RDBMS-kogemuste ja tööriistade maksimaalne kasutamine

MongoDB on selgelt eelistatav järgmistel juhtudel:

- Semistruktureeritud või varieeruva skeemiga andmed
- Suuremahulised kirjutamisoperatsioonid, kus järjepidevus ei ole absoluutne nõue
- Geograafiliselt hajutatud süsteemid, mis vajavad lihtsat horisontaalset skaleerimist
- Reaalajas rakendused (IoT, logid, sündmused)
- Kiire prototüüpimine ja agiilne arendus muutuva skeemiga

Neutraalsed stsenaariumid (kumbki võib sobida):

- Veebisaidid ja blogid standardse sisu haldamisega
- Kasutajate autentimis- ja profiilisüsteemid
- Standardne e-kaubanduse kataloog ilma keerukate seosteta

6.1.3 RK3: Andmemudeli mõju arendusele

Andmemudeli erinevused mõjutavad arendustööd mitmel viisil:

Skeemi muutused: MongoDB paindlik skeem teeb arendustsükli skeemi muutmise lihtsamaks – ei ole vaja andmebaasi migratsiooni faile hallata. See on eelis kiires arenduses, kuid võib viia andmete ebajärjepidevusele tootmises, kui muutusi ei hallata hoolikalt. PostgreSQL nõuab eksplitsiitseid migratsioonifaile (näiteks Alembic Pythoni kontekstis), mis on tülikam, kuid tagab skeemi järjepidevuse.

ORM ja andmemudelid: Mõlemate andmebaaside jaoks on saadaval laialdased teegid. PostgreSQL kontekstis kasutatakse sageli SQLAlchemy (Python), Hibernate (Java) või ActiveRecord (Ruby). MongoDB jaoks on olemas Mongoose (Node.js), Motor (Python) ja Spring Data MongoDB. Küpsuse ja võimaluste poolest on PostgreSQL ORM-id üldjuhul küpsemad.

Päringute keerukus: SQL on deklaratiivne ja intuitiivne keerukate päringute jaoks. MongoDB aggregatsioonipipeline on võimas, kuid keerulisem õppida ja siluda. Lihtsamate operatsioonide jaoks on MongoDB API intuitiivsem JSON-põhise arenduse kontekstis.

6.1.4 RK4: Hübriidarhitektuurid

Fowleri ja Sadalage [9] kontseptsioon “polyglot persistence” on praktikas laialt levinud. Käesoleva töö põhjal on soovituslik hübriidmuster järgmine:

1. **PostgreSQL** tehingupõhiste, kriitiliste andmete jaoks (kasutajate kontod, tellimused, maksed)
2. **MongoDB** dokumentiandmete, sündmuste logide ja muutuva skeemiga andmete jaoks
3. **Redis** vahemälu ja seansiandmete jaoks
4. **Elasticsearch** täistekstotsingu ja analüütika jaoks

See lähenemine on kooskõlas mikroteenuste arhitektuuri põhimõtetega, kus iga teenus valib andmesäilituslahenduse oma vajadustest lähtuvalt.

6.2 Otsustuspuu andmebaasi valikuks

Joonis 6.1 illustreerib soovituslikku otsustuspuud andmebaasi valikul.

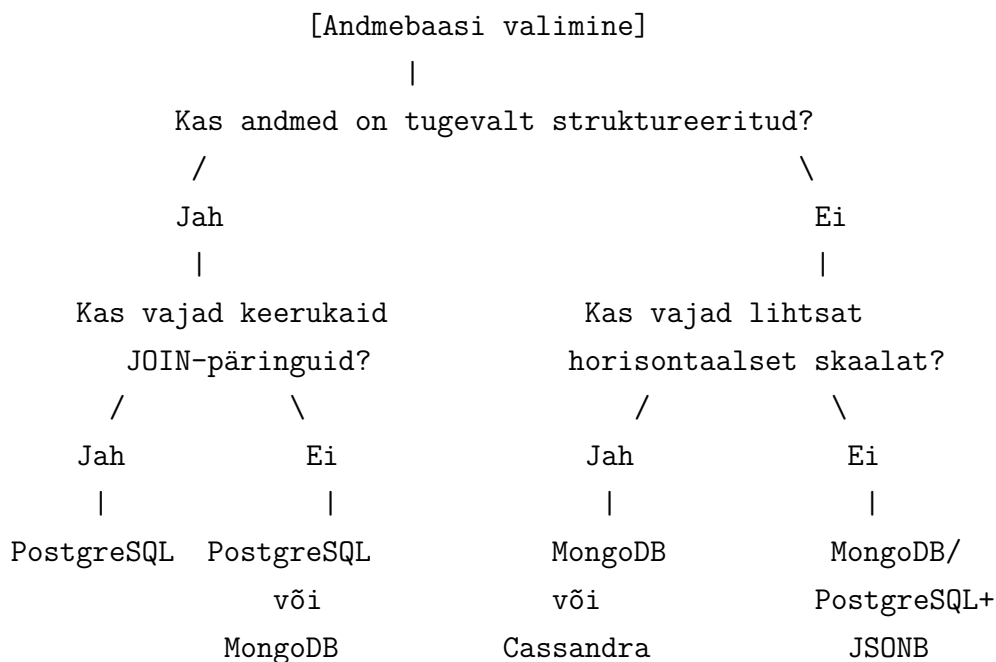


Figure 6.1: Lihtsustatud otsustuspuu andmebaasi valikuks

6.3 Piirangud ja ohud

6.3.1 Uurimistöö piirangud

Käesoleval uurimistööl on mitu piirangut, mida tulemuste tõlgendamisel tuleb arvesse võtta:

1. **Sünteesilised andmed:** Kuigi andmekogumid olid kavandatud reaalseid kasutusstsenaariumeid imiteerima, ei pruugi need täpselt peegeldada kõigi

tootmissüsteemide andmemustreid.

2. **Üks serverimääratlus:** Testid viidi läbi üksikul serveril. Klaster- ja replika-konfiguratsioonide mõju ei ole käesolevas töös uuritud.
3. **Versioonide areng:** Andmebaasisüsteemid arenevad kiiresti ning järgmised versioonid võivad jõudlusomadusi oluliselt muuta.
4. **Konfiguratsioonisõltuvus:** Jõudlus sõltub suuresti konfiguratsioonist. Käesolevas töös kasutati mõistlikke tootmissätteid, kuid optimeerimise ruumi on rohkem.

6.3.2 Levinud vead andmebaasi valikul

Praktika põhjal saab välja tuua mitu levinud viga:

- **Trend-põhine valik:** MongoDB valimine lihtsalt sellepärast, et see on “kaasaegne” või vastupidi PostgreSQL valimine harjumuse tõttu, ilma nõudeid analüüsimata.
- **Jõudluse üle hindamine:** Jõudlus on sageli ülehinnatud otsustegur. Mõlema süsteemi jõudlus on piisav enamiku rakenduste jaoks, kui seda õigesti häälestada.
- **Skeemi puudumine MongoDB-s:** Harjumus jätta MongoDB dokumentidele igasugune struktuur puudub võib viia andmeprobleemideni tootmises.
- **JOIN-de ignoreerimine MongoDB valikul:** Kui andmed on loomuliselt relatsioonilised (palju seoseid), ei ole MongoDB denormalisatsioon alati sobilik.

Chapter 7

Kokkuvõte ja tulevikuperspektiivid

7.1 Peamised järeldused

Käesolev bakalaureusetöö uuris PostgreSQL relatsioonilist andmebaasihaldussüsteemi ja MongoDB dokumentorienteeritud andmebaasihaldussüsteemi, et selgitada välja nende tugevused, nõrkused ja sobivad kasutusstsenaariumid veebirakenduste kontekstis.

Läbiviidud jõudlustestid ja teoreetiline analüüs võimaldavad teha järgmised peamised järeldused:

1. **Pole universaalselt paremat lahendust.** Kummagi süsteemi eelis sõltub kasutusstsenaariumist, andmemudelist ja nõuetest. Küsimus pole “PostgreSQL või MongoDB”, vaid “milline süsteem sobib paremini minu konkreetse ülesande jaoks”.
2. **ACID versus paindlikkus.** PostgreSQL pakub tugevamaid garantiisid andmete tervikluse kohta, mis on kriitiliste rakenduste jaoks asendamatu. MongoDB paindlikum skeem on eelis kiires arenduses ja semistruktureeritud andmete korral.
3. **Jõudluserinevused on kontekstisõltuvad.** Testitava operatsiooni tüüp, andmemudeli kujundus (normaliseeritud vs. denormaliseeritud) ja konfiguratsioon mõjutavad jõudlust rohkem kui andmebaasiparadigma ise. Samade andmekindluse garantiidega konfigureeritud süsteemide jõudlus on üldjuhul sarnane.
4. **MongoDB on teinud suure arengu.** Mitmesuse tehingute lisamine MongoDB 4.0-s on kõrvaldanud ühe olulisima puuduse võrreldes relatsiooniliste süsteemidega. Tänapäevane MongoDB sobib paljudele stsenaariumidele, mis

varem vajasid tingimata relatsioonilist andmebaasi.

5. **Hübriidlahendused on praktikas levinud.** Mikroteenuste arhitektuurides kasutatakse sageli mitmeid andmesäilituslahendusi korraga, valides igaühe vastavalt teenuse vajadustele.

7.2 Soovitused praktikutele

Uurimistöö põhjal saab anda järgmised praktilised soovitused:

Kasutage PostgreSQL-i, kui:

- Teie andmed on tugevalt relatsioonilised (palju seoseid tabelite vahel)
- Teil on keerukad aruandlusnõuded
- Andmete terviklus on kriitilise tähtsusega
- Teie meeskond on SQL-iga harjunud
- Teil on vaja keerukaid tehinguid mitme tabeli üleselt

Kasutage MongoDB-d, kui:

- Teie andmemudel on dokumentilaadne ja hierarhiline
- Teil on vaja sagedasi skeemimuutusi arendustsüklis
- Teil on vaja horisontaalset skaleerimist pilvekeskkonnas
- Kirjutamine on domineeriva mahtudega (logid, sündmused, IoT)
- Teie andmed on semistruktureeritud varieeruva struktuuriga

7.3 Tulevikusuundumused

Andmebaaside valdkond areneb kiiresti. Mõned olulisemad suundumused, mis mõjutavad PostgreSQL ja MongoDB kasutamist lähiaastatel:

7.3.1 Konvergennts

Huvitav tendents on kahe paradigma lähenemine teineteise poole:

- PostgreSQL on lisanud tugeva JSONB-toe, mis võimaldab dokumentilaadset salvestust relatsioonilises andmebaasis
- MongoDB on lisanud ACID-tehingud ja muutunud rohkem relatsioonilikuks

See konvergennts muudab valikuotsuse lähitulevikus veelgi keerulisemaks.

7.3.2 Pilvepõhised lahendused

Pilveteenused nagu AWS Aurora (PostgreSQL-ühilduv), Azure Cosmos DB (MongoDB-ühilduv) ja Google Cloud Spanner pakuvad horisontaalset skaleeritavust ilma traditsiooniliste NoSQL-kompromissideta. Need nn. NewSQL-süsteemid kombineerivad relatsioonilist mudelit hajutatud arhitektuuriga.

7.3.3 Suurandmed ja tehisintellekt

Tehisintellekti ja masinõppe levimine mõjutab ka andmebaaside valikut. Vektorandmebaasid (pgvector PostgreSQL jaoks, MongoDB Atlas Vector Search) muutuvad oluliseks AI-rakenduste kontekstis, kus on vaja semantilist otsingut.

7.3.4 Serverless ja edge computing

PlanetScale, Turso ja Neon pakuvad serverless andmebaasiteenuseid, mis töötavad väljalaskelähedal (edge). See uus paradigma mõjutab nii PostgreSQL-põhiseid kui MongoDB-põhiseid lahendusi.

7.4 Uurimistöö edasiarendused

Käesolev töö avab mitmeid huvitavaid uurimissuundi:

- Klaster-konfiguratsioonide võrdlus (MongoDB Sharding vs. PostgreSQL Citus)
- NewSQL süsteemide (CockroachDB, TiDB) kaasamine võrdlusesse
- Süvitsi uuring konkreetse tööstusharu kontekstis (tervishoid, e-kaubandus)
- Arendajate produktiivsuse mõõtmine kvalitatiivsete meetoditega
- Energiatõhususe võrdlus (andmebaasi ökoloogiline jalajälg)

Kokkuvõttes annab käesolev bakalaureusetöö süstemaatilise ja empiiriliselt põhjendatud ülevaate PostgreSQL ja MongoDB tugevatest ja nõrkadest külgedest ning pakub praktilisi juhiseid andmebaasi valikuks. Lootuses, et see töö aitab eesti keeles kättesaadavaks teha andmebaaside valdkonna olulise teadmuse ning toetab nii tudengite kui praktikute informeeritud otsuste tegemist.

Kirjanduse loend

- [1] Veronika Abramova and Jorge Bernardino. “NoSQL Databases: MongoDB vs Cassandra”. In: *Proceedings of the International Conference on Computer Science and Software Engineering* (2013), pp. 14–22.
- [2] Eric A. Brewer. “Towards Robust Distributed Systems”. In: *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. Portland, Oregon: ACM, 2000, p. 7.
- [3] Rick Cattell. “Scalable SQL and NoSQL Data Stores”. In: *ACM SIGMOD Record* 39.4 (2011), pp. 12–27. DOI: [10.1145/1978915.1978919](https://doi.org/10.1145/1978915.1978919).
- [4] Fay Chang et al. “Bigtable: A Distributed Storage System for Structured Data”. In: *ACM Transactions on Computer Systems* 26.2 (2008), pp. 1–26. DOI: [10.1145/1365815.1365816](https://doi.org/10.1145/1365815.1365816).
- [5] Kristina Chodorow. *MongoDB: The Definitive Guide*. 2nd. Sebastopol, CA: O’Reilly Media, 2013. ISBN: 978-1449344689.
- [6] Edgar F. Codd. “A Relational Model of Data for Large Shared Data Banks”. In: *Communications of the ACM* 13.6 (1970), pp. 377–387. DOI: [10.1145/362384.362685](https://doi.org/10.1145/362384.362685).
- [7] C. J. Date. *An Introduction to Database Systems*. 8th. Boston, MA: Addison-Wesley, 2004. ISBN: 978-0321197849.
- [8] Giuseppe DeCandia et al. “Dynamo: Amazon’s Highly Available Key-Value Store”. In: *Proceedings of 21st ACM SIGOPS Symposium on Operating Systems Principles*. Stevenson, Washington: ACM, 2007, pp. 205–220. DOI: [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281).
- [9] Martin Fowler and Pramod J. Sadalage. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley Professional, 2012. ISBN: 978-0321826626.
- [10] Seth Gilbert and Nancy Lynch. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”. In: *ACM SIGACT News* 33.2 (2002), pp. 51–59. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).

- [11] Cornelia Györödi et al. “A Comparative Study: MongoDB vs. MySQL”. In: *Proceedings of the 13th International Conference on Engineering of Modern Electric Systems (EMES)*. IEEE, 2015, pp. 1–6. DOI: [10.1109/EMES.2015.7158433](https://doi.org/10.1109/EMES.2015.7158433).
- [12] Theo Haerder and Andreas Reuter. “Principles of Transaction-Oriented Database Recovery”. In: *ACM Computing Surveys* 15.4 (1983), pp. 287–317. DOI: [10.1145/289.291](https://doi.org/10.1145/289.291).
- [13] Karamjit Kaur and Rinkle Rani. “Modeling and Querying Data in NoSQL Databases”. In: *Proceedings of the IEEE International Conference on Big Data* (2013), pp. 1–7.
- [14] Neal Leavitt. “Will NoSQL Databases Live Up to Their Promise?” In: *IEEE Computer* 43.2 (2010), pp. 12–14. DOI: [10.1109/MC.2010.58](https://doi.org/10.1109/MC.2010.58).
- [15] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 2012. ISBN: 978-0321826626.
- [16] Michael Stonebraker and Lawrence A. Rowe. “The Design of POSTGRES”. In: *ACM SIGMOD Record* 15.2 (1986), pp. 340–355. DOI: [10.1145/16856.16888](https://doi.org/10.1145/16856.16888).
- [17] Michael Stonebraker et al. “The End of an Architectural Era: (It’s Time for a Complete Rewrite)”. In: *Proceedings of the 33rd International Conference on Very Large Data Bases*. VLDB Endowment, 2007, pp. 1150–1160.

Appendix A

Testimisskriptid

A.1 PostgreSQL jõudlustestide Python-skript

```
1 #!/usr/bin/env python3
2 """
3 PostgreSQL jõudlustesti skript
4 Kasutamine: python3 pg_benchmark.py --host localhost --n
5             100000
6 """
7 import time
8 import psycopg2
9 import random
10 import string
11 import statistics
12 import argparse
13 from dataclasses import dataclass
14 from typing import List
15
16 @dataclass
17 class TestResult:
18     test_name: str
19     iterations: int
20     times_ms: List[float]
21
22     @property
23     def median_ms(self) -> float:
24         return statistics.median(self.times_ms)
25
```

```
26     @property
27     def stdev_ms(self) -> float:
28         return statistics.stdev(self.times_ms) if len(self.
times_ms) > 1 else 0.0
29
30     @property
31     def p95_ms(self) -> float:
32         sorted_times = sorted(self.times_ms)
33         idx = int(len(sorted_times) * 0.95)
34         return sorted_times[min(idx, len(sorted_times) - 1)]
35
36 def generate_user_data(n: int) -> List[dict]:
37     """Genereerib n kasutaja testiandmed."""
38     users = []
39     for i in range(n):
40         users.append({
41             'kasutajanimi': f'user_{i:08d}',
42             'email': f'user{i}@test.example.com',
43             'eesnimi': ''.join(random.choices(string.
ascii_letters, k=8)),
44             'perenimi': ''.join(random.choices(string.
ascii_letters, k=10)),
45             'vanus': random.randint(18, 80),
46         })
47     return users
48
49 def test_single_insert(conn, users: List[dict],
iterations: int = 10) -> TestResult:
50
51     """Test T1:  ksikute  kirjete lisamine."""
52     times = []
53     cursor = conn.cursor()
54
55     for _ in range(iterations):
56         # Puhastame tabeli enne iga iteratsiooni
57         cursor.execute("DELETE FROM kasutajad WHERE
kasutajanimi LIKE 'user_%'")
58         conn.commit()
59
60         start = time.perf_counter()
61         for user in users:
62             cursor.execute(
```

```
63         """INSERT INTO kasutajad
64             (kasutajanimi, email, eesnimi, perenimi,
vanus)
65             VALUES (%s, %s, %s, %s, %s)""" ,
66         (user['kasutajanimi'], user['email'],
67         user['eesnimi'], user['perenimi'], user['
vanus'])
68     )
69     conn.commit()
70     elapsed = (time.perf_counter() - start) * 1000
71     times.append(elapsed)
72
73     cursor.close()
74     return TResult("T1: Unikaarne INSERT", len(users),
times)
75
76 def test_batch_insert(conn, users: List[dict],
77                       batch_size: int = 1000) -> TResult:
78     """Test T2: Partiip hine sisestamine."""
79     times = []
80     cursor = conn.cursor()
81
82     for _ in range(5):
83         cursor.execute("DELETE FROM kasutajad WHERE
kasutajanimi LIKE 'user_%'")
84         conn.commit()
85
86         start = time.perf_counter()
87         for i in range(0, len(users), batch_size):
88             batch = users[i:i + batch_size]
89             cursor.executemany(
90                 """INSERT INTO kasutajad
91                     (kasutajanimi, email, eesnimi, perenimi,
vanus)
92                     VALUES (%s, %s, %s, %s, %s)""" ,
93                 [(u['kasutajanimi'], u['email'],
94                 u['eesnimi'], u['perenimi'], u['vanus'])
95                 for u in batch]
96             )
97             conn.commit()
98             elapsed = (time.perf_counter() - start) * 1000
```



```

99         times.append(elapsed)
100
101     cursor.close()
102     return TResult("T2: Partii INSERT", len(users), times)
103
104 def test_primary_key_lookup(conn, ids: List[int],
105                             iterations: int = 10) ->
106     TResult:
107         """Test T3: Primaarv tme alusel lugemine."""
108         times = []
109         cursor = conn.cursor()
110
111         for _ in range(iterations):
112             start = time.perf_counter()
113             for pk_id in random.sample(ids, min(1000, len(ids))):
114                 cursor.execute(
115                     "SELECT * FROM kasutajad WHERE id = %s", (
116                         pk_id,)
117                 )
118                 _ = cursor.fetchone()
119                 elapsed = (time.perf_counter() - start) * 1000
120                 times.append(elapsed)
121
122             cursor.close()
123             return TResult("T3: Primaarv tme lugemine",
124                             min(1000, len(ids)), times)
125
126 def print_results(results: List[TestResult]):
127     """Kuvab testitulemused tabelina."""
128     print(f"\n{'='*70}")
129     print(f"{'Test':<35} {'Mediaanlat.':<15} {'Std.h lve':<12} {'p95':<10}")
130     print(f"{'='*70}")
131     for r in results:
132         print(f"{r.test_name:<35} {r.median_ms:>10.1f} ms "
133               f"{r.stdev_ms:>8.1f} ms {r.p95_ms:>8.1f} ms")
134     print(f"{'='*70}\n")
135
136 def main():
137     parser = argparse.ArgumentParser()
138     parser.add_argument('--host', default='localhost')

```

```
137     parser.add_argument('--port', type=int, default=5432)
138     parser.add_argument('--db', default='testdb')
139     parser.add_argument('--user', default='postgres')
140     parser.add_argument('--n', type=int, default=10000,
141                         help='Andmepunktide arv')
142     args = parser.parse_args()
143
144     conn = psycopg2.connect(
145         host=args.host, port=args.port,
146         dbname=args.db, user=args.user
147     )
148
149     print(f"Genereeritakse {args.n} testikirjet...")
150     users = generate_user_data(args.n)
151
152     results = []
153     print("K iivitame Test T1...")
154     results.append(test_single_insert(conn, users[:1000]))
155
156     print("K iivitame Test T2...")
157     results.append(test_batch_insert(conn, users))
158
159     # Saame ID-d lugemistestiks
160     cursor = conn.cursor()
161     cursor.execute("SELECT id FROM kasutajad LIMIT 10000")
162     ids = [row[0] for row in cursor.fetchall()]
163     cursor.close()
164
165     print("K iivitame Test T3...")
166     results.append(test_primary_key_lookup(conn, ids))
167
168     print_results(results)
169     conn.close()
170
171 if __name__ == '__main__':
172     main()
```

Listing A.1: PostgreSQL jõudlustesti põhiskript

Appendix B

Andmebaaside konfiguratsiooni võrdlustabel

Table B.1: PostgreSQL ja MongoDB põhifunktsioonide võrdlus

Funktsioon	PostgreSQL	MongoDB
Andmemudel	Relatsioonitabelid	BSON-dokumendid
Skeem	Rangelt struktureeritud	Paindlik (schema-less)
Päringukeel	SQL	MQL (MongoDB Query Language)
ACID-tehingud	Täielik tugi	Tugi alates v4.0
Liitmised	JOIN-operaatorid	\$lookup aggregatsiooniga
Skaleerumine	Põhiliselt vertikaalne	Horisontaalne (sharding)
Replitseerimine	Streaming Replication	Replica Set
Indeksid	B-tree, GiST, GIN jt	B-tree, geospatial jt
Täistekstotsing	tsvector/tsquery	Tekst-indeksid
JSON-tugi	JSONB (natiivne)	Natiivne (BSON)
Ajakirjastamine	WAL	Ajakiri (journal)
Autentimine	Rollipõhine (RBAC)	Rollipõhine (RBAC)
Litsents	PostgreSQL License (vaba)	SSPL (osaliselt piiratud)
Esmane kasutusjuht	OLTP, analüütika	Dokumendisäilitus, skaalamine
Tüüpilised kasutajad	Finants, ERP, CMS	Veeb, IoT, big data

Appendix C

Kasutatud testimisandmekogumite statistika

Table C.1: Testimisandmekogumite kirjeldus

Andmekogum	Kirjete arv	PG suurus	Mongo suurus	Kirjeldus
DS1	1 000	0.5 MB	0.3 MB	Väike
DS2	10 000	4.8 MB	3.1 MB	Väike-keskmine
DS3	100 000	47 MB	31 MB	Keskmine
DS4	1 000 000	468 MB	312 MB	Suur
DS5	10 000 000	4.6 GB	3.1 GB	Väga suur